

lab2

要求：跟lab1差不多意思，但是要实现这些测试样例：

`test_wait` `test_clone` `test_fork` `test_execve` `test_getppid` `test_exit` `test_yield`
`test_waitpid` `test_gettimeofday` `test_sleep`

▼ 实验前置：

- 我们在init.c里添加这几个测试样例名：

```
char *tests[] = {
    "getcwd",
    "write",
    "getpid",
    "times",
    "uname",

    // lab2

    "wait",
    "clone",
    "fork",
    "execve",
    "getppid",
    "exit",
    "yield",
    "waitpid",
    "gettimeofday",
    "sleep",
};
```

- 执行make fast_renew：
 - 我在Makefile里新添加了这个用法，不知道逻辑对不对，是跟着助教的视频一步一步抠出来的，每次git push之前都make fast_renew一下，提交到平台上也是正常有分的。

```
fast_renew:
    @make clean
    @make build
    @make dump
    @make fs
    @make clean
    @make run
```

- 然后观察输出，会看到有这么几个不明的系统调用号，查询 https://github.com/oscomp/testsuite-for-oskernel/blob/main/oscomp_syscalls.md 这个文档，我们可以找到这几个调用号对应的定义：

- 220: `#define SYS_clone 220`
- 260: `#define SYS_wait4 260`
- 173: `#define SYS_getppid 173`
- 169: `#define SYS_gettimeofday 169`

```
===== START test_wait =====
pid 7 wait: unknown sys call 220
pid 7 wait: unknown sys call 260
--- Assert Fatal ! ---
init: starting clone
===== START test_clone =====
pid 8 clone: unknown sys call 220

--- Assert Fatal ! ---
init: starting fork
===== START test_fork =====
pid 9 fork: unknown sys call 220

--- Assert Fatal ! ---
init: starting execve
===== START test_execve =====
    I am test_echo.
execve success.
===== END main =====
```

```

init: starting getppid
===== START test_getppid =====
pid 11 getppid: unknown sys call 173
    getppid error.
===== END test_getppid =====
init: starting exit
===== START test_exit =====
pid 12 exit: unknown sys call 220

    --- Assert Fatal ! ---
init: starting yield
===== START test_yield =====
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 260
pid 13 yield: unknown sys call 260
pid 13 yield: unknown sys call 260
===== END test_yield =====
init: starting waitpid
===== START test_waitpid =====
pid 14 waitpid: unknown sys call 220

    --- Assert Fatal ! ---
init: starting gettimeofday
===== START test_gettimeofday =====
pid 15 gettimeofday: unknown sys call 169
pid 15 gettimeofday: unknown sys call 169
gettimeofday error.
===== END test_gettimeofday =====
init: starting sleep
===== START test_sleep =====
pid 16 sleep: unknown sys call 169

    --- Assert Fatal ! ---

```

- 在 `sysnum.h` 中添加这些调用号以后，在 `syscall.c` 中进行函数声明：

```
extern uint64 sys_clone(void);
extern uint64 sys_wait4(void);
extern uint64 sys_getppid(void);
extern uint64 sys_gettimeofday(void);
```

进行系统调用号对应函数指针的系统调用表补充：

```
[SYS_clone] sys_clone,
[SYS_wait4] sys_wait4,
[SYS_getppid] sys_getppid,
[SYS_gettimeofday] sys_gettimeofday,
```

进行名字表的对应：

```
[SYS_clone] "clone",
[SYS_wait4] "wait4",
[SYS_getppid] "getppid",
[SYS_gettimeofday] "gettimeofday",
```

- 别忘了在 `usys.pl` 中为这些函数名注册，否则用户态在链接时，就不知道要怎么链接函数名了：

```
entry("clone");
entry("getppid");
entry("wait4");
entry("gettimeofday");
```

- 接下来就可以挨个函数进行实现了。

▼ wait (wait4):

- 在 `sysproc.c` 这个文件中，我们搜索一下 `wait`，看看有没有已经实现的内容。在之后的所有样例实现之前，我都会搜索一下，看看我们要做的是仅仅对齐系统调用号还是要重头定义并实现一个函数呢。

```
uint64
sys_wait(void)
{
    uint64 p;
```

```

    if (argaddr(0, &p) < 0)
        return -1;
    return wait(p);
}

```

- 这里已经给出了一个 `wait` 的函数，其实这个就是测试仓库里之前查到的 `wait4` 那个调用号对应的实现效果，我们也把它完全复制过来，然后改一下函数名，获得我们自己的 `wait4()` 函数。
- 再看看具体需要我们做什么：

```

// wstatus表示当前子进程的退出状态
int cpid, wstatus;
cpid = fork();
if(cpid == 0){
    printf("This is child process\n");
    exit(0);
}
else{
    pid_t ret = wait(&wstatus);
    assert(ret != -1);
    if(ret == cpid)
        printf("wait child success.\nwstatus: %d\n",
wstatus);
    else
        printf("wait child error.\n");
}

```

- 可以看到这个测试样例的运行过程是这样：先进行 `fork`，父进程中将子进程的 `pid` 存放在 `cpid` 变量中，而子进程的运行从 `fork()` 下一句开始，子进程的 `cpid=0`；在 `printf`（这个过程会比较慢，所以会切换到父进程的运行中）之后，子进程 `exit(0)` 结束；在父进程中，`ret` 为调用 `wait(&wstatus)` 的返回值，当父进程等待到这个子进程状态变化，返回的 `pid` 后，如果这个 `pid==cpid`，则测试成功。
- 看一下最关键的 `wait(&wstatus)` 到底是怎么定义的吧：

```

int wait(int *code)
{

```

```
return waitpid((int)-1, code, 0);
```

又套了一个waitpid:

```
int waitpid(int pid, int *code, int options)
{
    return syscall(SYS_wait4, pid, code, options,
0);
}
```

发现确实最后都是使用的 `SYS_wait4` 这个系统调用。看看这两个函数的逻辑，`wait()` 传的参数为指向 `wstatus` 变量的指针，这个指针会作为 `waitpid()` 的第二个参数 `code`，并作为 `syscall` 传递给内核的第三个参数 `code`。

- 然后我们再回到内核态，也就是我们自己要实现的函数这里来，这里也有一个 `wait(p)` 函数，看一看是啥：

```
// 等待一个子进程exit，并返回其pid
// Return -1 如果这个进程没有子进程
int wait(uint64 addr)
{
    struct proc *np;
    int havekids, pid;
    struct proc *p = myproc(); // 这里是获取当前的进程，调用了myproc

    // 获取当前进程（父进程）的锁，父进程会在后面的整个循环中一直持有这个锁
    // 直到子进程退出时尝试唤醒父进程
    // 父进程会阻塞子进程，直到它进入sleep()
    acquire(&p->lock);

    for (;;)
    {
        // 查询全局进程表proc，找自己的子进程，np指针指向proc数组头，直至到最大限额NPROC为止
        havekids = 0;
        for (np = proc; np < &proc[NPROC]; np++)
```

```

    {
        // this code uses np->parent without holding np
        ->lock.
        // acquiring the lock first would cause a deadl
        ock,
        // since np might be an ancestor, and we alread
        y hold p->lock.
        // 检查哪个进程是p的子进程np
        if (np->parent == p)
        {
            // np->parent can't change between the check
            and the acquire()
            // because only the parent changes it, and w
            e're the parent.
            acquire(&np->lock);
            havekids = 1;
            // 找到kid之后, 如果这个子进程的状态是ZOMBIE, 则父进
            程为其处理后事
            if (np->state == ZOMBIE)
            {
                // Found one.
                pid = np->pid;
                if (addr != 0 && copyout2(addr, (char *)&np
                ->xstate, sizeof(np->xstate)) < 0)
                {
                    // 如果一开始给wait函数的传参地址是可以使用
                    的, 那就把这个僵尸子进程的退出状态码xstate
                    // 粘进去, 然后释放子进程的锁, 释放父进程的锁
                    release(&np->lock);
                    release(&p->lock);
                    // 粘贴失败就返回-1
                    return -1;
                }
                // 彻底清除np进程的页表, 解除trapframe等等
                freeproc(np);
                release(&np->lock);
                release(&p->lock);
                // 粘贴成功, 返回子进程pid
            }
        }
    }

```

```

        return pid;
    }
    release(&np->lock);
}
}

// No point waiting if we don't have any children.
if (!havekids || p->killed)
{
    release(&p->lock);
    return -1;
}

// 查了一圈以后发现没有子进程退出了，就把父进程sleep，让
被阻塞的子进程继续运行
sleep(p, &p->lock); // DOC: wait-sleep
}
}

```

- 然后这里提到了一些有关进程的结构体，我们不妨直接看一看，这样后面处理任何有关进程的东西，就不会一抹黑了。

- `struct proc`：进程结构体

```

struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    struct proc *parent;           // Parent process
    void *chan;                     // If non-zero,
    sleeping on chan                // sleeping on chan
    int killed;                     // If non-zero,
    have been killed                // have been killed
    int xstate;                     // Exit status to
    be returned to parent's wait
    int pid;                        // Process ID
}

```



```

    // these are private to the process, so p->lock
    need not be held.
    uint64 kstack;                // Virtual address
    ss of kernel stack
    uint64 sz;                    // Size of process
    memory (bytes)
    pagetable_t pagetable;        // User page table
    pagetable_t kpagetable;       // Kernel page table
    struct trapframe *trapframe; // data page for
    trampoline.S
    struct context context;        // swtch() here
    to run process
    struct file *ofile[NFILE];    // Open files
    struct dirent *cwd;            // Current directory
    char name[16];                // Process name
    (debugging)
    int tmask;                    // trace mask
};

```

- `myproc()` : 从cpu中获取当前进程的指针

```

struct proc *
myproc(void)
{
    push_off();
    struct cpu *c = mycpu();
    struct proc *p = c->proc;
    pop_off();
    return p;
}

```

- `struct proc proc[ NPROC ]` : 全局进程表 `proc`
- 回归正题，我们现在需要实现的 `wait4` 到底是什么功能？在 `wait.c` 的测试样例中，我们知道，用户态 `wait` 的本质实际是在调用一个传入 `pid=int(-1)` 的

`waitpid`，而`waitpid`其实就是等待一个指定 `pid` 的进程结束。那么我们现有的内核态 `wait(uint64 addr)` 就不够看了，我们起码要添加一个参数，这个参数是指定等待的子进程 `pid`，也就是由用户态 `syscall(SYS_wait4, pid, code, options, 0)`；这一句传来的第二个参数。

- 调整一下我们内核态的`wait()`，让它符合这个要求：`int wait(uint64 addr, int wait_pid)`
- 对了，你还得记着调整`kernel/include/proc.h`里的声明奥：`int wait(uint64, int)`；不然编译会报错的。
- 只需要微调一下现有函数的内容，使之符合这个判断的逻辑：

```
if (np->parent == p)
{
    // np->parent can't change between the check
    and the acquire()
    // because only the parent changes it, and w
    e're the parent.
    acquire(&np->lock);

    if (wait_pid == -1 || np->pid == wait_pid)
    {
        havekids = 1;
    }

    if (np->state == ZOMBIE)
    {
        ...
    }
}
```

- 随后在`sysproc.c`里调整自带的`sys_wait`函数，以及我们的`sys_wait4`函数：

```
uint64
sys_wait(void)
{
    uint64 p;
    if (argaddr(0, &p) < 0)
        return -1;
    return wait(p, -1);
}
```

这里你要注意：系统自带的 `wait` 函数对应的是正常情况下用户态的 `wait` 调用，因此第一个参数也就是 `wstatus` 的指针，但我们自己的 `wait4` 要注意到用户态的 `waitpid` 实际上第二个参数才是 `wstatus` 的指针，所以在这里要对顺序进行调整，第一个参数是 `pid` 才对

```
uint64
sys_wait4(void)
{
    uint64 addr;
    int wait_pid;
    if (argint(0, &wait_pid) < 0 || argaddr(1, &addr) <
    0)
        return -1;
    return wait(addr, wait_pid);
}
```

▼ clone:

- 这个在 `sysproc.c` 里没有，我们得自己实现一个
- 看看我们该做什么：

```
size_t stack[1024] = {0};
static int child_pid;

static int child_func(void){
    printf("  Child says successfully!\n");
    return 0;
}

void test_clone(void){
    TEST_START(__func__);
    int wstatus;
    child_pid = clone(child_func, NULL, stack, 1024,
    SIGCHLD);
    assert(child_pid != -1);
    if (child_pid == 0){
        exit(0);
    }else{
```

```

        if(wait(&wstatus) == child_pid)
            printf("clone process successfully.\npid:%d\n", child_pid);
        else
            printf("clone process error.\n");
    }

    TEST_END(__func__);
}

```

- 就是要执行 `clone()`，在child进程（因为clone和fork有所区分）中执行 `exit`，然后再检查父进程的 `wait` 是否等到了自己 `clone` 出来的child进程的 `exit`
- 但这里的stack，child_func都只是很像形式上的占位，但是如果我们再看看clone_test.py，就发现这个child_func是需要被正确执行才可以的，但是如果我们只是给sys_clone()套壳成fork()，是拿不到测试点全部的3分的
- 看一下所谓的clone在用户态是怎么定义的：

```

pid_t clone(int (*fn)(void *arg), void *arg, void
*stack, size_t stack_size, unsigned long flags)
{
    if (stack)
        stack += stack_size;    // 由于地址增长
        放下向低地址，这里要还回去，变成堆顶指针

    return __clone(fn, stack, flags, NULL, NULL, N
ULL);
    //return syscall(SYS_clone, fn, stack, flags,
    NULL, NULL, NULL);
}

```

- 这里我觉得应该把第一个 `return` 注释掉，用第二个 `return`，这样我们才能正常的让它链接到我们自己的内核态 `clone` 逻辑？其实并不是的，这里的 `__clone`其实是一个非常精妙的汇编封装，我们在 `testsuits-for-oskernel/riscv-syscalls-testing/user/src/oscomp/clone.s`找到了有关 `__clone`的功能：

```

# __clone(func, stack, flags, arg, ptid, tls, ctid)
#          a0,    a1,    a2,  a3,   a4,  a5,   a6

# syscall(SYS_clone, flags, stack, ptid, tls, ctid)
#          a7    a0,    a1,   a2,  a3,   a4

.global __clone
.type __clone, %function
__clone:
    # Save func and arg to stack
    addi a1, a1, -16
    sd a0, 0(a1)
    sd a3, 8(a1)

    # Call SYS_clone
    mv a0, a2
    mv a2, a4
    mv a3, a5
    mv a4, a6
    li a7, 220 # SYS_clone
    ecall

    beqz a0, 1f
    # Parent
    ret

    # Child
1:    ld a1, 0(sp)
    ld a0, 8(sp)
    jalr a1

    # Exit
    li a7, 93 # SYS_exit
    ecall

```

我们不需要读太明白这个汇编代码的意思，需要注意的是上面人为写的注释，这告诉我们传参的情况，以及这个汇编封装到底做了什么。

- 首先最关键的一步，在整段__clone开始的第一个部分中，它将a0和a3的参数存进了a1参数为地址头的第0个和第8个位置，我们看一下具体的逻辑：

```
# __clone(func, stack, flags, arg, ptid, tls, c
tid)
#          a0,    a1,    a2,  a3,    a4,  a5,
a6

# syscall(SYS_clone, flags, stack, ptid, tls, c
tid)
#          a7      a0,    a1,    a2,  a3,
a4

.global __clone
.type __clone, %function
__clone:
    # Save func and arg to stack
    addi a1, a1, -16    # 这里先把指向stack的栈顶
                        # 指针向低地址移动了16字节，相当于开辟了一块16字节的栈空间
    sd a0, 0(a1)        # 将第一个参数func（一个8
                        # 字节大小的函数指针）放在栈的开头
    sd a3, 8(a1)        # 将第四个参数arg放在func
                        # 后面的栈空间里
```

- 随后就是调整参数位置，执行ecall指令并进入SYS_clone的系统调用。其实在后面这里，相当于运行了一次 `syscall(SYS_clone, flags, stack, ptid, tls, ctid)` 指令，那么我们就不能在 `trapframe` 中再直接取到 `func` 参数了，那就不能直接执行这个 `func`
- 然后在 `ecall` 执行完毕返回这个汇编封装后，parent和child各自执行自己的逻辑，这里是经典的分流操作，如果a0是子进程，也就是 `beqz`（是否等于0）为真，则跳转至前方的1标签处

```
beqz a0, 1f          # 检查ecall系统调用SYS_clone以
                    # 后，a0作为系统调用的返回值是什么
```

parent: 直接 `ret` 了

child: 从栈指针的0字节位置取出, 并存入`a1`, 从8字节位置取出并存入`a0`, 然后再跳转执行`a1`中的指令, 最后再系统调用一次 `exit` 就结束啦

```
1:      ld a1, 0(sp)
        ld a0, 8(sp)
        jalr a1

        # Exit
        li a7, 93 # SYS_exit
        ecall
```

- 然后我们回到内核态, 在助教给的文档中, 说到了 `clone` 和 `fork` 其实只有细微的差别, 但是差别是什么我们先不管, 先看看真正的 `fork` 是啥样的:

```
int fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // 这个allocproc的逻辑其实就是搜索全局进程表proc (我们在wait里分析过了), 找一个能用的进行初始化
    if ((np = allocproc()) == NULL)
    {
        return -1;
    }

    // 将父进程的页表全部真正拷贝到一块新的np进程空间中
    if (uvmcopy(p->pagetable, np->pagetable, np->kpagetable, p->sz) < 0)
    {
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    // 子进程与父进程size一样
```

```

np->sz = p->sz;
    // 设置父进程为p
np->parent = p;

// copy tracing mask from parent.
np->tmask = p->tmask;

// 由于子进程与父进程共享同样的用户态上下文，因此直接全部粘过去
*(np->trapframe) = *(p->trapframe);

// 为了区分系统调用结束后谁是子进程（注意这个fork就是一次系统调用），
// 把子进程的a0强行返回0，父进程的a0则返回子进程的pid
np->trapframe->a0 = 0;

// 共享并增加一次打开文件的计数器
for (i = 0; i < NOFILE; i++)
    if (p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = edup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));

pid = np->pid;

// 现在子进程可以被运行，并释放锁
np->state = RUNNABLE;

release(&np->lock);

return pid;
}

```

- 然后阅读一下[clone \(2\) - Linux 手册页面](#) --- [clone\(2\) - Linux manual page](#)，有一大堆很复杂的形容，大致意思就是 `clone` 可以更精细地通过 `flags` 参数进行选择，是软拷贝还是硬拷贝父进程的一些结构等等的。

- 不过我们管他那么多的，我们根本不用处理 `flags`，我们只需要在 `clone` 之后子进程能够直接执行 `child_func` 不就得了。
- 不过在此之前，我们先试试看如果直接用 `fork` 的逻辑原封不动地跑这个样例会怎么样：

```
uint64
sys_clone(void)
{
    return fork();
}
```

```
init: starting clone
===== START test_clone =====
usertrap(): unexpected scause 0x0000000000000002 p
id=10 clone
sepc=0x0000000000000000 stval=0x0000000000000000
clone process successfully.
pid:10
===== END test_clone =====
```

发现报错了：sepc=0表示引发异常的那条指令的地址，而scause=2则表示这是执行了非法指令导致的报错。也就是说在子进程结束SYS_clone以后返回汇编封装时，它执行了那两条从栈顶取数据的指令，但是在跳转a1并执行时，发现这个在0x0处的指令完全是非法的，这是因为在0x0的地址处，大部分都是空映射0，而RISCV的指令集编码中，全0是不属于任何有效指令的。

- 其实这个原因很明确，我们的 `__clone` 本来只是把 `func` 这条指令存到了 `stack` 变量中，这个 `stack` 是最后我们希望在 `ecall` 结束并返回后，`sp` 指向的位置，这个 `sp` 指向这里的话，就会从 `stack` 的顶端得到我们想要的 `func` 指令，然后再正常执行 `child_func`，完成 `printf` 通过测试点，可是传统的 `fork` 逻辑里，

```
// 由于子进程与父进程共享同样的用户态上下文，因此直接全部粘过去
*(np->trapframe) = *(p->trapframe);
```

这一步把所有的父进程 `trapframe` 全都给子进程了，子进程的 `sp` 这时候实际上指向的仍然是父进程的旧栈，而不是我们要的 `stack` 变量顶，所以在最后 `ecall` 结束返回时，`sp` 就只能从父进程的旧栈读数据，然后就报错了

- 然后我们的操作就是，从

```
# syscall(SYS_clone, flags, stack, ptid, tls, ctid)
#          a7      a0,    a1,   a2,   a3,   a4
# Call SYS_clone
    mv a0, a2
    mv a2, a4
    mv a3, a5
    mv a4, a6
    li a7, 220 # SYS_clone
    ecall
```

这一部分，找到我们的 `stack` 变量究竟被存在哪一个参数里，交给了我们的内核，其实显而易见就在 `a1` 里面了，然后我们就可以添加这样一个逻辑（注意其他的代码都是直接复制 `fork` 的代码），这样就可以实现我们的要求

```
*(np->trapframe) = *(p->trapframe);    // 这是原本fork
的代码
// xv6中，如果调用clone传了有效的stack地址，则必须重置子进程
栈顶和相关的a1参数！
    if (stack != 0)
    {
        np->trapframe->sp = stack;
    }
np->trapframe->a0 = 0;                    // 这是原本fork
的代码
```

然后在 `proc.c` 中，我们的 `clone` 函数长成这个样子：

```
int clone(uint64 stk)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();
    uint64 stack = stk;
```

```

...    // 同于fork()
*(np->trapframe) = *(p->trapframe);

// 除了这个if, 其他都是fork的原封不动
if (stack != 0)
{
    np->trapframe->sp = stack;
}

np->trapframe->a0 = 0;
...
}

```

在 `proc.h` 里声明我们这个函数 `int clone(uint64 stack);` 这一步有点像前面的 `wait` 的操作, 最后在 `sysproc.c` 里, 把我们的 `sys_clone` 完善一下就OK了

```

uint64
sys_clone(void)
{
    uint64 stack;
    if (argaddr(1, &stack) < 0)
        return -1;
    return clone(stack);
}

```

▼ fork:

好像正常实现了wait之后, fork直接就通过了

▼ execve:

- 查了一下发现 `sysproc.c` 里有这个实现, 然后看看我们该做什么:

```

void test_execve(void){
    TEST_START(__func__);
    char *newargv[] = {"test_echo", NULL};
    char *newenviron[] = {NULL};
    execve("test_echo", newargv, newenviron);
    printf("  execve error.\n");
}

```

```

        //TEST_END(__func__);
    }

```

虽然说这里我们什么都不用管，直接 `make fast_renew` 就能通过，但是我们还是看看这个传参逻辑吧：

- 看一下 `execve` 在用户态的实现，就是一个很简单的传参，然后执行 `syscall` 进行系统调用

```

int execve(const char *name, char *const argv[], c
har *const argp[])
{
    return syscall(SYS_execve, name, argv, argp);
}

```

这里有一个情况，我们在用户端调用的分明是 `SYS_execve` 信号，为啥到了内核态，却是由 `SYS_exec` 来接手下面的操作呢？这是因为在用户端，如果我们查询这个 `SYS_evecve` 信号的定义，它其实是一个对应221数字的调用号，而我们在内核态，只是看到有人在调用221这个调用号，那在内核态来看，不就是我们定义的 `#define SYS_exec 221` 吗，那就用这个 `SYS_exec` 及其对应的函数什么的来响应好了

- 那我们看看内核态的 `execve` 函数吧

```

uint64
sys_exec(void)
{
    char path[FAT32_MAX_PATH], *argv[MAXARG];
    int i;
    uint64 uargv, uarg;

    // 非常正常的取参数过程，跟用户态的syscall调用完全可以对齐
    if (argstr(0, path, FAT32_MAX_PATH) < 0 || argadr(1, &uargv) < 0)
    {
        return -1;
    }
    memset(argv, 0, sizeof(argv));
    // 循环搬运参数数组，把这些以NULL结尾的字符串指针一个一

```

个搬过来

```
for (i = 0;; i++)
{
    if (i >= NELEM(argv))
    {
        goto bad;
    }
    if (fetchaddr(uargv + sizeof(uint64) * i, (uint64 *)&uarg) < 0)
    {
        goto bad;
    }
    if (uarg == 0)
    {
        argv[i] = 0;
        break;
    }
    argv[i] = kalloc();
    if (argv[i] == 0)
        goto bad;
    if (fetchstr(uarg, argv[i], PGSIZE) < 0)
        goto bad;
}
// 包装了一下，把用户态的参数都规规矩矩处理好，交给exec
真正进行执行
int ret = exec(path, argv);

for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
    kfree(argv[i]);

return ret;

bad:
for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
    kfree(argv[i]);
```

```
    return -1;
}
```

- 接下来的exec具体是啥我们就不看了，总之是执行了“test_echo”这个可执行文件，得到了我们需要的测试样例结果

```
#include "stdio.h"

/*
 * for execve
 */

int main(int argc, char *argv[]){
    printf(" I am test_echo.\nexecve success.\n");
    TEST_END(__func__);
    return 0;
}
```

▼ getppid:

- 看看具体做什么吧

```
int test_getppid()
{
    TEST_START(__func__);
    pid_t ppid = getppid();
    if(ppid > 0) printf(" getppid success. ppid : %d\n", ppid);
    else printf(" getppid error.\n");
    TEST_END(__func__);
}
```

就只需要得到>0的 `ppid` 就行了，甚至你都不用 `getppid`，直接调用 `getpid` 都通过了，这里会得到 `pid=14`

但是我们还是修改一下ppid的逻辑，万一后面有用

- 编写一个 `sys_ppid` 函数，这就可以了，这里会得到 `ppid=1`，其实就是我们的 `init.c` 得到的初始进程，这也比较明显，因为 `init.c` 中，我们执行循环，然

后 `fork` 出来其他的进程，我们总共进行了9个测试样例，到这里是第10个，从1往后数，这是第11个，但是我们在 `clone` 那看到，这时 `clone` 出来的子进程占用了 `pid=10`（之前的 `wait` 也占用了2个pid），随后 `fork` 样例执行后又占用了两个 `pid`，到这里 `getppid` 测试样例的进程，正好到 `pid=14` 了

```
uint64
sys_getppid(void)
{
    return myproc()->parent ? myproc()->parent->pid :
    0;
}
```

▼ exit:

- 好像跟wait.c要求差不多？能正常退出就行

```
void test_exit(void){
    TEST_START(__func__);
    int cpid, waitret, wstatus;
    cpid = fork();
    assert(cpid != -1);
    if(cpid == 0){
        exit(0);
    }else{
        waitret = wait(&wstatus);
        if(waitret == cpid) printf("exit OK.\n");
        else printf("exit ERR.\n");
    }
    TEST_END(__func__);
}
```

这个exit在内核态我们就不用额外实现了，`sysproc.c` 里面的 `sys_exit` 也就是对 `exit` 做了一个提取参数的前置套壳工作，`exit()` 函数在 `proc.c` 里有实现，我们就不多说了，直接可以正常通过测试点

▼ yield:

- 如果我们直接执行make fast_renew，会看到这里抛出这样的报错

```

init: starting yield
===== START test_yield =====
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
===== END test_yield =====

```

去查一下手册，看看这个124是什么调用号，发现对应在用户态的，这个是 `#define SYS_sched_yield 124`，那我们就需要在内核态这一端，也声明一个124的系统调用号 `#define SYS_yield 124`，然后再进行一圈注册，回到 `sysproc.c`里，现在有一个纯为了能通过编译的函数 `sys_yield()`

- 看看我们需要做啥

```
/*
理想结果：三个子进程交替输出
*/

int test_yield(){
    TEST_START(__func__);

    for (int i = 0; i < 3; ++i){
        if(fork() == 0){
            for (int j = 0; j < 5; ++j){
                sched_yield();
                printf(" I am child process: %d. iteration %d.\n", getpid(), i);
            }
            exit(0);
        }
    }
    wait(NULL);
    wait(NULL);
    wait(NULL);
    TEST_END(__func__);
}
```

fork出来3个子进程，然后每个子进程都进行一次sched_yield，如果理想，我们能得到15行交替的输出

- 然后我们查一查 `proc.c`，经过之前fork、clone样例，我们知道这个脚本下面存着很多现成的函数，这些函数基本上都是经过 `sysproc.c` 处理传参问题，然后最终调用的底层函数，我们发现确实有 `yield`

```
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
}
```

```

    release(&p->lock);
}

```

它主要还是获取了当前进程的锁，并设置当前进程状态为 `RUNNABLE` 而非 `RUNNING`，这就是让出进程的前置操作，然后进行了 `sched()`，这个 `sched` 主要是检查当前的状态是不是可以进行进程切换，然后交给外部定义的 `swtch` 切换进程，这个样例就结束了，所以我们最终只需要得到一个这样的 `sys_yield`

```

uint64
sys_yield(void)
{
    yield();
    return 0;
}

```

▼ waitpid:

- 看看我们该做什么

```

int i = 1000;
void test_waitpid(void){
    TEST_START(__func__);
    int cpid, wstatus;
    cpid = fork();
    assert(cpid != -1);
    if(cpid == 0){
        while(i--);
        sched_yield();
        printf("This is child process\n");
        exit(3);
    }else{
        pid_t ret = waitpid(cpid, &wstatus, 0);
        assert(ret != -1);
        if(ret == cpid && WEXITSTATUS(wstatus) == 3)
            printf("waitpid successfully.\nwstatus: %
x\n", WEXITSTATUS(wstatus));
        else
            printf("waitpid error.\n");
    }
}

```

```

    }
    TEST_END(__func__);
}

```

这个又很像之前的wait要求我们做的事情，只不过这里强行把子进程的退出码设成3了，如果实现了yield和wait，这里我们完全不用进行任何操作，因为这里的系统调用函数 `waitpid` 实际上底层也是进行 `wait4` 的 `syscall`

```

int waitpid(int pid, int *code, int options)
{
    return syscall(SYS_wait4, pid, code, options, 0);
}

```

▼ gettimeofday:

- 看看我们该做什么

```

/*
 * 测试通过时的输出:
 * "gettimeofday success."
 * "start:[num], end:[num]"
 * "interval: [num]"    注: 数字[num]的值应大于0
 * 测试失败时的输出:
 * "gettimeofday error."
 */
void test_gettimeofday() {
    TEST_START(__func__);
    int test_ret1 = get_time();
    volatile int i = 12500000; // qemu时钟频率1250000
    0
    while(i > 0) i--;
    int test_ret2 = get_time();
    if(test_ret1 > 0 && test_ret2 > 0){
        printf("gettimeofday success.\n");
        printf("start:%d, end:%d\n", test_ret1, test_
ret2);
        printf("interval: %d\n", test_ret2 -
test_ret1);
    }
}

```

```

    }else{
        printf("gettimeofday error.\n");
    }
    TEST_END(__func__);
}

```

可以看到调用了两次系统调用函数 `get_time()`，第一次调用以后，经过一个 qemu 时钟周期后再次调用，最终要检查这两次调用得到的结果相减，看看是否间隔 > 0 ，这样才算完全通过，如果你在这里只是让 `sys_gettimeofday` 返回 0，就只能拿到 2 分，具体为什么，可以看一下 `gettimeofday_test.py`，它最后有一行 `interval` 与 0 的大小比较

- 看看核心函数是怎样的

```

int64 get_time()
{
    TimeVal time;
    int err = sys_get_time(&time, 0);
    if (err == 0)
    {
        return ((time.sec & 0xffff) * 1000 + time.
usec / 1000);
    }
    else
    {
        return -1;
    }
}

int sys_get_time(TimeVal *ts, int tz)
{
    return syscall(SYS_gettimeofday, ts, tz);
}

```

如果系统调用后，不产生 err，则返回一个 TimeVal 结构体中的内容，我们看看这个 TimeVal 是啥

```

typedef struct
{

```

```

uint64 sec; // 自 Unix 纪元起的秒数
uint64 usec; // 微秒数
} TimeVal;

```

那我们在内核态也应该实现这样的一个结构体，并且往里面填东西

- 发现 `syscall.c` 和 `proc.c` 中都没有我们想要的现成函数，只能自己实现一下，不过在此之前，我们先了解一下系统中内核态的time、tick与真实时间的关系
 - `time`：RISC-V中硬件的 `mtime` 寄存器，只要一开机，就会按照固定的物理频率雷打不动一直+1，这个+1的速度，就是上面提到的，qemu主板的时钟频率12,500,000（12.5MHz），也就是说真实时间过1s，`mtime` 寄存器会增加12,500,000
 - `tick`：这是内核态自己维护的整数变量，定义在 `timer.c` 中，内核并不像 `mtime` 那样实时更新 `tick`，而是定了一个闹钟，这个闹钟是设定好的宏 `INTERVAL`，等 `mtime` 过了这么久以后，产生一个 *TimerInterrupt* 中断，CPU停止手里的工作，进入到 `trap.c` 然后是 `timer.c`，执行 `timer_tick`，为ticks+1，随后设定下一次中断的时间，假如设置 `INTERVAL` 使得 `tick` 在每秒可触发100次时钟中断，那一个 `tick` 就是真实时间1/100秒

```

struct spinlock tickslock;
uint ticks;

void timerinit() {
    initlock(&tickslock, "time");
}

void
set_next_timeout() {
    sbi_set_timer(r_time() + INTERVAL);
}

void timer_tick() {
    acquire(&tickslock);
    ticks++;
    wakeup(&ticks);
    release(&tickslock);
}

```

```

        set_next_timeout();
    }

```

- 之后，其实有这么一个函数r_time()，是作为查看 `mtime` 的函数接口使用的，自此，我们就可以使用r_time()获取硬件时间，然后经过真实时间的换算，得到确切的时间了

```

uint64
sys_gettimeofday(void)
{
    uint64 addr;
    if (argaddr(0, &addr) < 0)
        return -1;

    uint64 t = r_time();
    struct
    {
        uint64 sec;
        uint64 usec;
    } tv;

    tv.sec = t / 12500000;
    tv.usec = (t % 12500000) * 1000000 / 12500000;

    if (copyout2(addr, (char *)&tv, sizeof(tv)) < 0)
        return -1;
    return 0;
}

```

▼ sleep

- 如果我们直接执行make fast_renew的话，会发现这样的报错

```

init: starting sleep
===== START test_sleep =====
pid 24 sleep: unknown sys call 101
-- Assert Fatal ! ---

```

不过这次我们不直接查手册了，我们直接看看我们应该做什么，也就是说测试样例会调用什么东西吧

- 看看我们要做什么

```
/*
 * 测试通过时的输出:
 * "sleep success."
 * 测试失败时的输出:
 * "sleep error."
 */
void test_sleep() {
    TEST_START(__func__);

    int time1 = get_time();
    assert(time1 >= 0);
    int ret = sleep(1);
    assert(ret == 0);
    int time2 = get_time();
    assert(time2 >= 0);

    if(time2 - time1 >= 1){
        printf("sleep success.\n");
    }else{
        printf("sleep error.\n");
    }
    TEST_END(__func__);
}
```

调用了 `get_time`，并观察两次得到的时间是否差距 ≥ 1 ，因为在中间，调用了核心的系统调用函数 `sleep(1)`，这里的 `get_time` 逻辑，我们已经在 `gettimeofday` 实现过了，我们看看这个 `sleep` 的具体实现

```
int sleep(unsigned long long time)
{
    TimeVal tv = {.sec = time, .usec = 0};
    if (syscall(SYS_nanosleep, &tv, &tv))
        return tv.sec;
```

```

    return 0;
}

```

然后直接查看 `SYS_nanosleep` 对应的系统调用号，发现 `#define SYS_nanosleep 101`，于是我们就知道，在内核态这一端，我也需要一个101的 `SYS_nanosleep` 调用号，不过在此之前，经过之前的 `SYS_sched_yield` 和 `SYS_yield` 的关系，我们还是留一个心眼，万一这里其实已经实现好了 `SYS_sleep` 这个调用号，只是它的号码没对齐为101呢？

- 。但其实并不行，因为实现好的 `sys_sleep` 的逻辑是这样的

```

uint64
sys_sleep(void)
{
    int n;
    uint ticks0;

    if (argint(0, &n) < 0)    // 取第一个参数，并将第一个
        return -1;           参数看做int整数
    acquire(&tickslock);
    ticks0 = ticks;
    while (ticks - ticks0 < n)    // 执行对第一个参数的
        sleep                  睡眠
        {
            if (myproc()->killed)
            {
                release(&tickslock);
                return -1;
            }
            sleep(&ticks, &tickslock);
        }
    release(&tickslock);
    return 0;
}

```

可是在我们 `syscall` 传参的时候，第一个参数 `&tv` 是一个uint64的地址指针，如果把它看做一个需要睡眠的时间 `n`，系统就像直接卡死在这了，因

为这个地址换算成十进制，差不多有上百亿大。因此我们还是需要自己实现一个 `sys_nanosleep`

- 然后我们注册好自己的 `sys_nanosleep`，并实现这个函数吧
 - 不过我们需要逻辑上真正的实现这个要求的秒+微秒的逻辑，所以在照搬 `sys_sleep` 之后，我们还需要计算一下这里 `ticks` 与真实时间秒的换算关系

```
uint64
sys_nanosleep(void)
{
    uint64 tvaddr;
    uint ticks0;

    if (argaddr(0, &tvaddr) < 0)
        return -1;

    struct TimeVal
    {
        uint64 sec;
        uint64 usec;
    } tv;
    // 把tv的地址当成uint64读出，而不是int
    if (copyin2((char *)&tv, tvaddr, sizeof(tv)) <
        0)
        return -1;

    acquire(&tickslock);
    ticks0 = ticks;
    // 这里要进行单位换算!
    while (ticks - ticks0 < tv.sec * 200 + tv.usec *
        200 / 1000000)
    {
        if (myproc()->killed)
        {
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
}
```

```

    release(&tickslock);
    return 0;
}

```

- 我们查询一下宏定义的 `INTERVAL`，看一看一秒内会产生多少个ticks（从这往下的这些小目录其实不太用看了，只要修改 `INTERVAL` 就行）

```

#define INTERVAL      (390000000 / 200)    // timer
interrupt interval (K210: 390MHz)

```

- 不过我们发现这个和我们之前在gettimeofday中提到的不一样啊？这里的分母并不是我们qemu用的“*qemu时钟频率12,500,000*”啊，那么这个更大的390,000,000是啥。查一下，我们知道这个390,000,000实际上是真实物理硬件K210的RISC-V双核芯片的频率**390MHz**，而我们在QEMU这个虚拟平台上模拟，为了节省宿主主机性能而设计的**12.5MHz**。
- 我想看看这个**12.5MHz**是在哪设定的，实际上这个并不是显式设定在C语言源码，而是QEMU自己写了一个虚拟主板，当运行 `qemu-system-riscv64 -machine virt ...` 这么一个指令时，这个QEMU在内存里凭空捏造了一块代号为 `virt` 的RISC-V主板，其参数会由设备树传递给操作系统内核

```

cpus {
#address-cells = <0x01>;
#size-cells = <0x00>;
timebase-frequency = <0x989680>;    // <--- 就是这里!!!

```

- 但是这里的 $0x989680 = 10,000,000$ 是**10MHz**啊，也不是代码里说的“*qemu时钟频率12,500,000*”，这是因为在官方的QEMU实现中，`timebase-frequency` 一直都是定死的10MHz，但在这里的xv6 K210实验中，早就把qemu-system-riscv64进行了魔改，因此目前我们使用的系统，它的 `mtime` 确实就是按照**12.5MHz**进行的，即使设备树里仍然是**10MHz**，这也就是为什么助教在测试样例中有这句声明

```

volatile int i = 12500000; // qemu时钟频率12500000

```

- 一句话：我们应该把 `INTERVAL` 改成，这样才能在逻辑上符合经修改后 QEMU 虚拟的 RISC-V 系统的逻辑

```
#define INTERVAL (12500000 / 200) // timer interrupt interval (QEMU: 12.5MHz)
```

- 不过实际上计算 ticks 与真实时间秒的关系，也用不上分母的数，我们看分子就行，分母的数是 `mtime` 的频率，分子才是一秒产生多少次时钟异常（产生多少个 ticks）
- 这还有一个事，虽然要求 `nanosleep`，但用户态调用的 `sleep` 那里，定义依然是 `usec`（微秒），而 $1\text{sec} = 1,000,000\text{usec}$ ，这也是为什么我们要进行 `/1,000,000` 操作